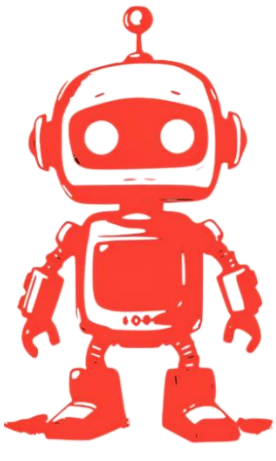
Coding Standards



AEGIS

bbd
bbdsoftware.com

Contents:

General

Since AEGIS uses a tech stack that primarily comprises of two languages as the foundation – Python and TypeScript – it is important that the language coding standards are followed throughout the development process. The goal of following these guidelines is to produce readable code, thus resulting in documentation that is easy to maintain.

Naming Conventions

TypeScript

- Use camelCase for standard variables and functions
  - `function funcName();`
  - `let myVar = 0;`
- Use PascalCase for components
  - `ComponentName({});`
- Use SCREAMING_SNAKE_CASE for constants
  - `const CONSTANT_NAME = 0;`

Python

- Use snake_case for functions and variables
  - `def function()`
  - `my_var = 0`
- Use PascalCase for classes
  - `ClassName`
- Use SCREAMING_SNAKE_CASE for constants
  - `CONST_NAME = 0`

Selection of Identifier names

- Use nouns to name variables and classes/components
- Use verbs to name functions
- Names should be descriptive of the purpose of the variable/function
- Avoid general names
- Avoid abbreviations, or single characters for names
- Booleans should use names that clearly indicate a Boolean value

Layout Rules

TypeScript

- Indent with tabs
- No whitespace at the end of or on blank lines
- Mathematical operators (except unary) should have space between operators
- No filler spaces in empty constructs
- Each source file must end with a new line
- Ensure all function bodies are indented
- Any **;** used as a statement terminator should be at the end of the line
- Use blank lines to separate parts of a function that are logically distinct
- Leave a blank line after a colon, semicolon or comma.

Python

- No whitespace at the end of or on blank lines
- Each source file must end with a new line
- Use exactly 4 columns worth of tab for indentation
- Do not mix tabs with spaces
- No blank line between a function and its decorator
- Group imports at the top of each file, with each import being on a separate line
- Lines must have a max length of 79 characters as per PEP 8 recommendation. Lines that exceed this length limit should wrap around
- Operators should be surrounded by a space on both sides
- Use blank lines to separate parts of a function that are logically distinct

Reliability

- Flake8 for Python linting and ESLint for TypeScript linting
- Code must be peer reviewed before merging to any shared branch
- Unit tests must test individual functions/features
- Integration between functions must be tested
- Test important user workflow
- Write thorough tests with well thought out edge cases
- Tests must be deterministic
- Inclusion of CodeCov that checks how much of the code was tested. Note, CodeCov should not be treated as proof that the code is correct; it only measures how much of the code was tested. Bad tests can still be inclusive.
- Inclusion of a GitHub Actions CI that runs the tests to ensure that the pushed code is not broken
- The CI automatically runs when a pull request is made to dev and main. Pull requests cannot be approved until all checks in the CI succeed.

Maintainability

- Use GitHub for version control, allowing us to keep an audit of who changed what and when
- Use SonarQube as an automated code review tool. This tool grades the quality of our code, highlighting maintainability and reliability issues before code is pushed to shared branches.
- Code should be organised into loosely coupled modules/components with clearly defined responsibilities
- Formatters ensure that code is easy to read in the future
- Thorough documentation helps the codebase stay easy to maintain

Commenting Practices

- The goal is to write self-explanatory code
- Comments must only explain what cannot plainly be expressed in the cod
- Explain the intent not the execution
- Make sure code and comments agree. Maintain comments as you refactor
- Do not write any obvious comments that restate obvious sections of code
- Keep comments brief

Security

- Secrets, API keys, tokens and other private credentials must not be committed to GitHub
- All external input must be validated in the client side and server side before usage
- Authentication checks must be performed on the server side
- Passwords must be encrypted before storage

Error handling

- Every try block should have a meaningful catch (and finally if necessary)
- Do not ignore silent errors
- Exceptions that are caught should provide enough context to diagnose the problem
- Server errors must be descriptive and use the correct HTTP status codes
- Exceptions should not expose sensitive information
- Users should be given error messages that indicate the next course of action
- User facing errors should not reveal internal implementation

Repository File Structure